

4.2 API Documentation

CampusCare Backend API Documentation

4.2.1 API Overview

The CampusCare backend is implemented using **Node.js** and **Express.js**. The mobile application communicates with the backend through RESTful API endpoints. Authentication is handled using **JWT tokens**, and protected endpoints require the token to be sent in the request header.

Base URL

```
http://192.168.1.206:3000/api
```

For local development, the IP address may change depending on the laptop network. The backend server runs on port:

```
3000
```

4.2.2 Authentication Requirements

Protected API endpoints require the following header:

```
{
  "Authorization": "Bearer JWT_TOKEN"
}
```

Some endpoints also require:

```
{
  "Content-Type": "application/json"
}
```

For image upload endpoints, the request must use:

```
multipart/form-data
```

4.2.3 Authentication APIs

1. Register User

Method and URL

POST /api/auth/register

Description

Registers a new user in the system. The user can register as a Community Member, Facility Manager, or Worker.

Authentication Required

No.

Allowed Roles

Public endpoint.

Request Headers

```
{
  "Content-Type": "application/json"
}
```

Request Body

```
{
  "name": "Test User",
  "email": "testuser@gmail.com",
  "password": "123456",
  "role": "community"
}
```

Body Parameters

Field	Type	Required	Description
name	string	Yes	Full name of the user
email	string	Yes	User email address
password	string	Yes	User password
role	string	Yes	User role: community, manager, or worker

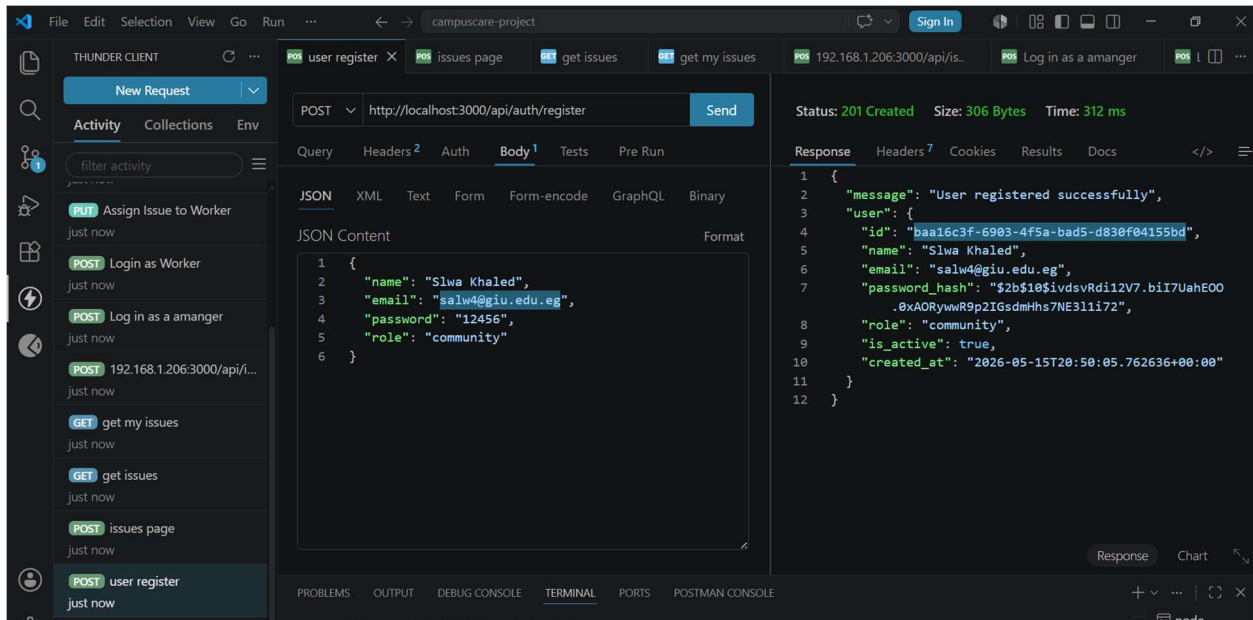
Success Response

```
{
  "message": "User registered successfully",
  "user": {
    "id": "uuid",
    "name": "Test User",
    "email": "testuser@gmail.com",
    "role": "community"
  }
}
```

```
}  
}
```

Error Response

```
{  
  "error": "Email already exists"  
}
```



2. Login User

Method and URL

POST `/api/auth/login`

Description

Authenticates the user using email and password. If the login is successful, the backend returns a JWT token and user information.

Authentication Required

No.

Allowed Roles

Public endpoint.

Request Headers

```
{  
  "Content-Type": "application/json"  
}
```

Request Body

```
{  
  "email": "manager1@gmail.com",  
  "password": "123456"  
}
```

Body Parameters

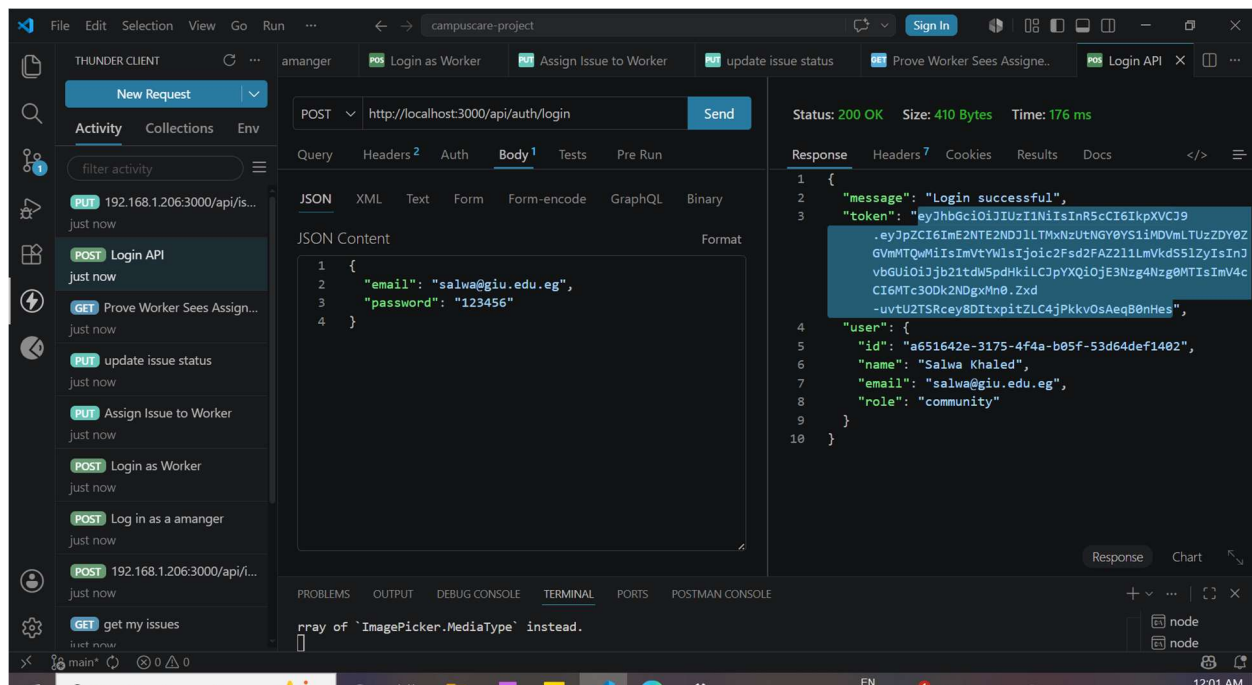
Field	Type	Required	Description
email	string	Yes	Registered user email
password	string	Yes	Registered user password

Success Response

```
{  
  "message": "Login successful",  
  "token": "JWT_TOKEN_HERE",  
  "user": {  
    "id": "uuid",  
    "name": "Manager Test",  
    "email": "manager1@gmail.com",  
    "role": "manager"  
  }  
}
```

Error Response

```
{  
  "error": "Invalid email or password"  
}
```



3. Logout User

Method and URL

POST /api/auth/logout

Description

Logs out the user. In the current implementation, logout is mainly handled on the frontend by removing the JWT token from local storage.

Authentication Required

Yes.

Allowed Roles

Community Member, Facility Manager, Worker.

Request Headers

```
{  "Authorization": "Bearer JWT_TOKEN"}
```

Success Response

```
{
  "message": "Logout successful"
}
```

Error Response

```
{
  "error": "Unauthorized"
}
```

4.2.4 Issue Management APIs

4. Submit New Issue

Method and URL

POST /api/issues

Description

Allows a Community Member to submit a new campus issue with title, description, category, location details, and optional issue photo.

Authentication Required

Yes.

Allowed Roles

Community Member.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN",
  "Content-Type": "multipart/form-data"
}
```

Request Body Example

```
title: Broken Chair
description: Chair broken in lecture hall
category: Furniture
building: B1
floor: 2
room: 202
photo: image-file.jpg
```

Body Parameters

Field	Type	Required	Description
title	string	Yes	Issue title
description	string	Yes	Issue description
category	string	Yes	Issue category
building	string	Yes	Building name/number
floor	string	Yes	Floor number
room	string	Yes	Room number
photo	file	Optional	Issue image

Success Response

```
{
  "message": "Issue created successfully with cloud photo upload",
  "issue": {
    "id": "uuid",
    "title": "Broken Chair",
    "description": "Chair broken in lecture hall",
    "category": "Furniture",
    "building": "B1",
    "floor": "2",
    "room": "202",
    "photo_url": "https://supabase-storage-url/image.jpg",
    "status": "Pending",
    "created_by": "user_uuid"
  }
}
```

Error Response

```
{
  "error": "Title and description are required"
}
```

5. Get All Issues

Method and URL

GET /api/issues

Description

Returns all submitted issues in the system. This endpoint is used by the Facility Manager dashboard.

Authentication Required

Yes.

Allowed Roles

Facility Manager, Manager, Admin.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN"
}
```

Success Response

```
{
  "message": "All issues retrieved successfully",
  "issues": [
    {
      "id": "uuid",
      "title": "Broken Chair",
      "description": "Chair broken in lecture hall",
      "category": "Furniture",
      "building": "B1",
      "floor": "2",
      "room": "202",
      "status": "In Progress",
      "created_by": "user_uuid",
      "assigned_to": "worker_uuid"
    }
  ]
}
```

Error Response

```
{
  "error": "Access denied. Your role is not allowed to perform this action."
}
```

6. Get My Issues

Method and URL

GET /api/issues/my

Description

Returns all issues submitted by the currently logged-in Community Member.

Authentication Required

Yes.

Allowed Roles

Community Member.

Request Headers

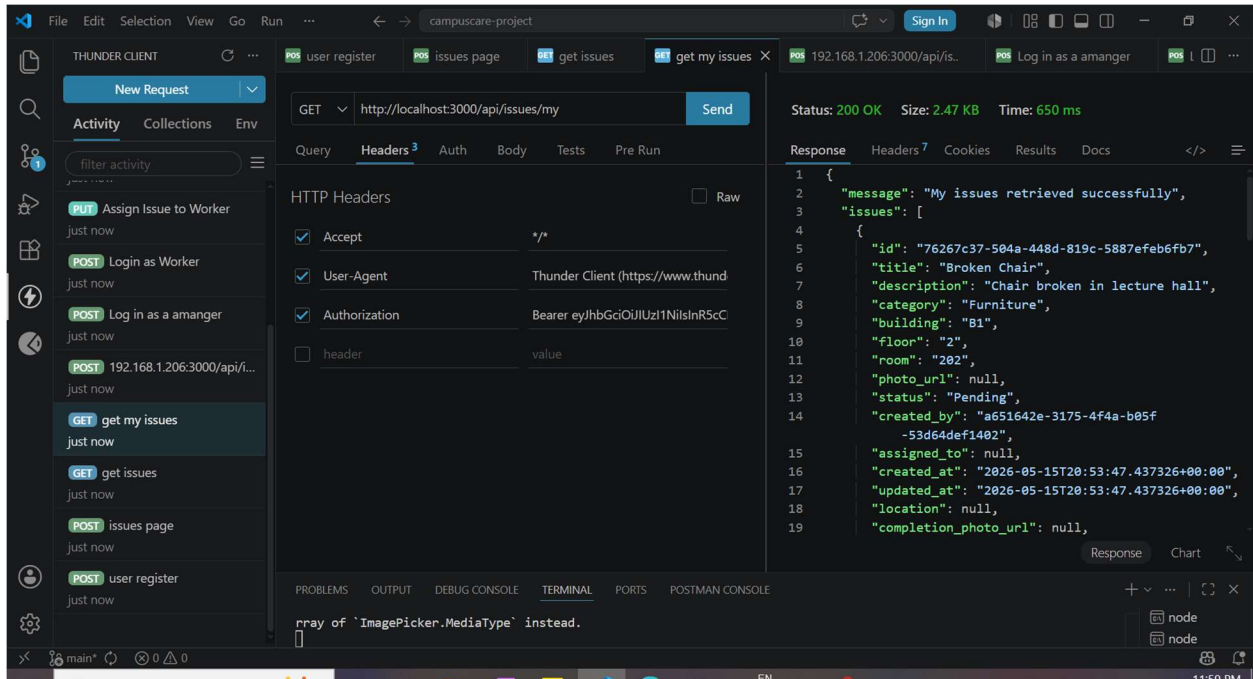
```
{  
  "Authorization": "Bearer JWT_TOKEN"  
}
```

Success Response

```
{  
  "message": "My issues retrieved successfully",  
  "issues": [  
    {  
      "id": "uuid",  
      "title": "Broken Table",  
      "category": "Furniture",  
      "status": "Pending",  
      "created_by": "community_user_uuid"  
    }  
  ]  
}
```

Error Response

```
{  
  "error": "Unauthorized"  
}
```



7. Get Issue Details

Method and URL

GET /api/issues/:id

Description

Returns full details of a specific issue.

Authentication Required

Yes.

Allowed Roles

Community Member, Facility Manager, Worker.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN"
}
```

URL Parameter

Parameter Type Description

id	uuid	Issue ID
----	------	----------

Example URL

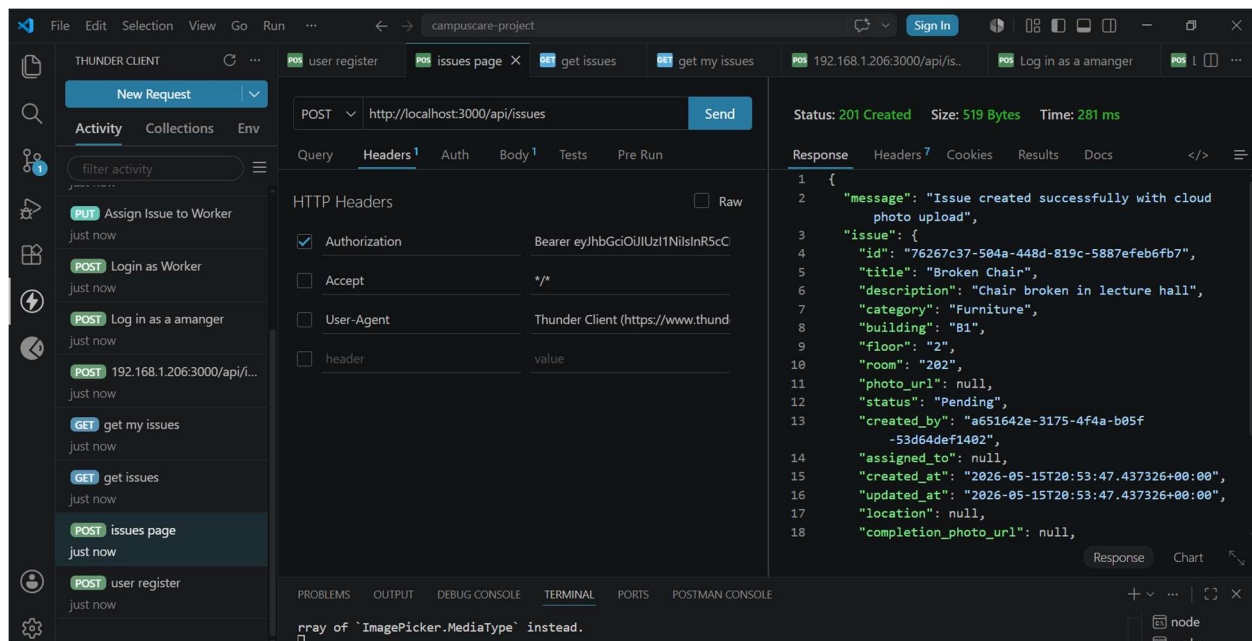
```
GET /api/issues/1e455aa3-db24-4b40-aed8-a18e7813e382
```

Success Response

```
{
  "message": "Issue details retrieved successfully",
  "issue": {
    "id": "1e455aa3-db24-4b40-aed8-a18e7813e382",
    "title": "Broken Chair",
    "description": "Chair broken in lecture hall",
    "category": "Furniture",
    "building": "B1",
    "floor": "2",
    "room": "202",
    "photo_url": "https://supabase-storage-url/photo.jpg",
    "status": "In Progress",
    "created_by": "community_user_uuid",
    "assigned_to": "worker_uuid",
    "completion_photo_url": null,
    "worker_comment": null
  }
}
```

Error Response

```
{
  "error": "Issue not found"
}
```



8. Update Issue Status

Method and URL

PUT /api/issues/:id/status

Description

Updates the status of an issue.

Authentication Required

Yes.

Allowed Roles

Facility Manager, Manager, Worker.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN",
  "Content-Type": "application/json"
}
```

Request Body

```
{
  "status": "In Progress"
}
```

Body Parameters

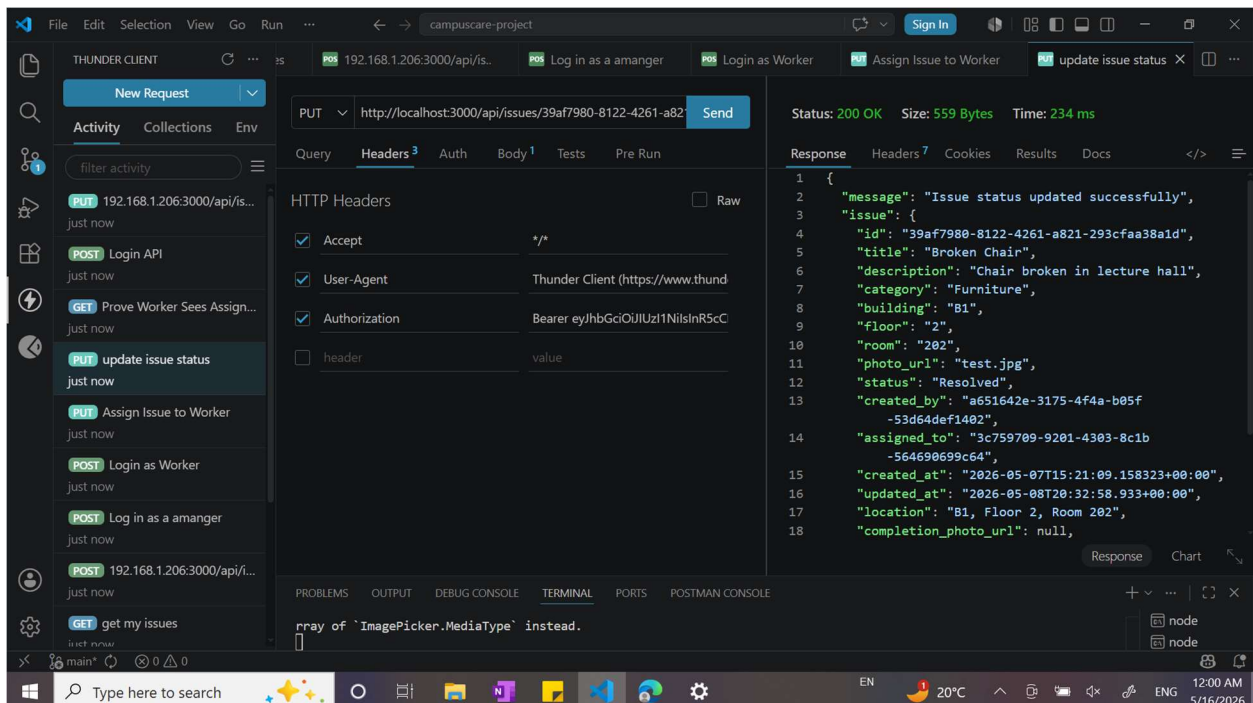
Field	Type	Required	Description
status	string	Yes	Issue status: Pending, In Progress, Resolved, Closed

Success Response

```
{
  "message": "Issue status updated successfully",
  "issue": {
    "id": "uuid",
    "status": "In Progress"
  }
}
```

Error Response

```
{
  "error": "Invalid status value"
}
```



9. Assign Issue to Worker

Method and URL

PUT /api/issues/:id/assign

Description

Allows the Facility Manager to assign an issue to a specific worker.

Authentication Required

Yes.

Allowed Roles

Facility Manager, Manager, Admin.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN",
  "Content-Type": "application/json"
}
```

Request Body

```
{
  "worker_id": "86745ed3-f2f2-4de6-aae1-c1942d7e6a4c"
}
```

Body Parameters

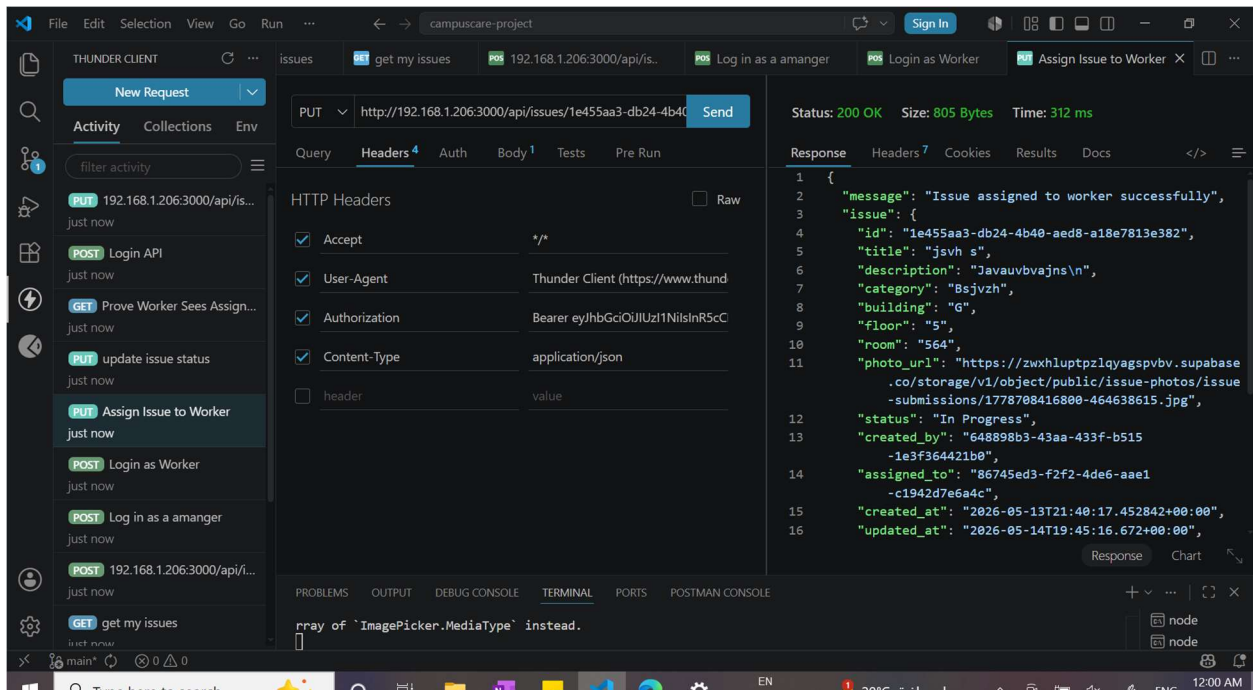
Field	Type	Required	Description
worker_id	uuid	Yes	ID of the worker assigned to the issue

Success Response

```
{
  "message": "Issue assigned to worker successfully",
  "issue": {
    "id": "1e455aa3-db24-4b40-aed8-a18e7813e382",
    "assigned_to": "86745ed3-f2f2-4de6-aae1-c1942d7e6a4c",
    "status": "In Progress"
  }
}
```

Error Response

```
{
  "error": "Access denied. Your role is not allowed to perform this action."
}
```



10. Close Issue

Method and URL

PUT /api/issues/:id/close

Description

Allows the Facility Manager to close an issue after it has been resolved.

Authentication Required

Yes.

Allowed Roles

Facility Manager, Manager, Admin.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN"
}
```

Success Response

```
{
  "message": "Issue closed successfully",
  "issue": {
    "id": "uuid",
    "status": "Closed"
  }
}
```

Error Response

```
{
  "error": "Issue not found"
}
```

11. Add Comment to Issue

Method and URL

POST /api/issues/:id/comments

Description

Allows a Worker to add a comment to an issue.

Authentication Required

Yes.

Allowed Roles

Worker.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN",
  "Content-Type": "application/json"
}
```

Request Body

```
{
  "comment": "Issue fixed successfully"
}
```

Body Parameters

Field	Type	Required	Description
comment	string	Yes	Worker comment

Success Response

```
{
  "message": "Comment added successfully",
  "comment": {
    "id": "uuid",
    "issue_id": "issue_uuid",
    "user_id": "worker_uuid",
    "comment": "Issue fixed successfully"
  }
}
```

Error Response

```
{
  "error": "Comment is required"
}
```

12. Upload Completion Photo

Method and URL

POST /api/issues/:id/photo

Description

Allows a Worker to upload a completion photo after resolving the assigned issue.

Authentication Required

Yes.

Allowed Roles

Worker.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN",
  "Content-Type": "multipart/form-data"
}
```

Request Body Example

worker_comment: Fixed successfully by worker
photo: completion-photo.jpg

Body Parameters

Field	Type	Required	Description
worker_comment	string	Optional	Worker explanation/comment
photo	file	Yes	Completion photo

Success Response

```
{
  "message": "Completion photo uploaded successfully and issue resolved",
  "issue": {
    "id": "uuid",
    "status": "Resolved",
    "completion_photo_url": "https://supabase-storage-url/completion-photo.jpg",
    "worker_comment": "Fixed successfully by worker"
  }
}
```

Error Response

```
{
  "error": "Completion photo is required"
}
```

13. Delete Issue

Method and URL

DELETE /api/issues/:id

Description

Allows the Facility Manager to delete an issue from the system.

Authentication Required

Yes.

Allowed Roles

Facility Manager, Manager, Admin.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN"
}
```

Success Response

```
{
  "message": "Issue deleted successfully"
}
```

Error Response

```
{
  "error": "Issue not found"
}
```

4.2.5 Worker APIs

14. Get Assigned Issues

Method and URL

GET /api/issues/assigned

Description

Returns all issues assigned to the currently logged-in worker.

Authentication Required

Yes.

Allowed Roles

Worker.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN"
}
```

Success Response

```
{
  "message": "Assigned issues retrieved successfully",
  "issues": [
```

```

{
  "id": "uuid",
  "title": "Laptop",
  "description": "Broken laptop in lecture hall",
  "category": "Technical",
  "building": "B3",
  "floor": "4",
  "room": "210",
  "status": "In Progress",
  "assigned_to": "worker_uuid"
}
]
}

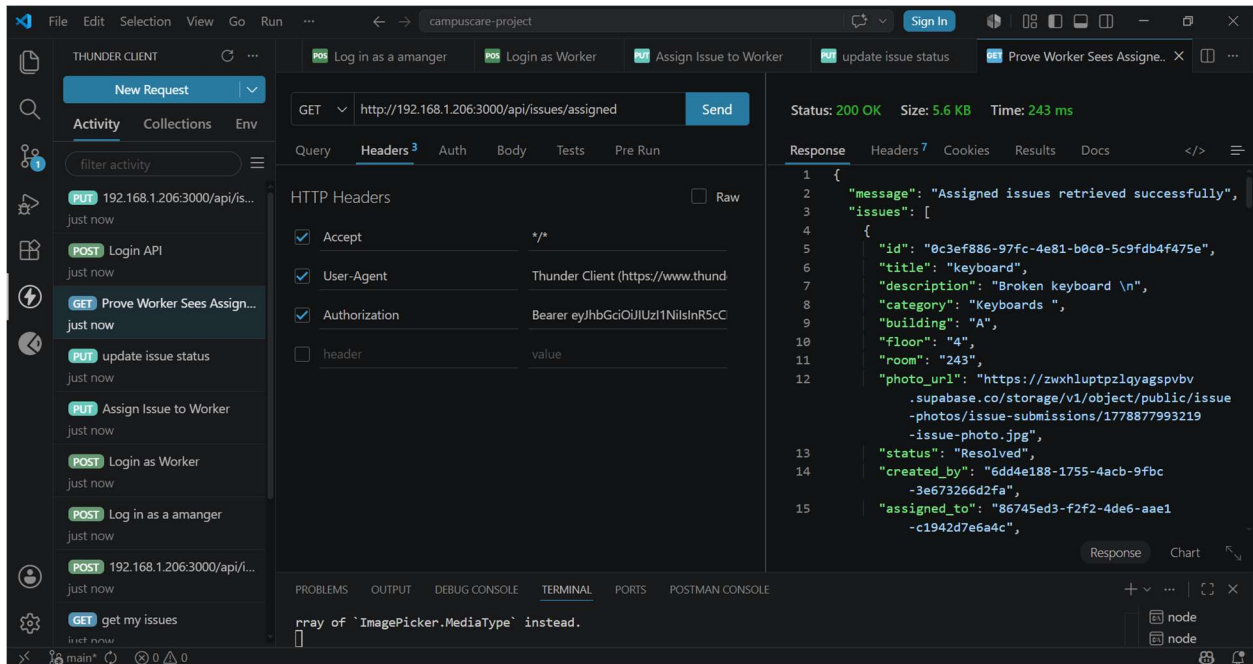
```

Error Response

```

{
  "error": "Access denied. Your role is not allowed to perform this action."
}

```



4.2.6 Manager Worker API

15. Get Available Workers

Method and URL

GET /api/issues/workers

Description

Returns the list of available users with the Worker role. This endpoint is used by the Facility Manager dashboard when assigning issues.

Authentication Required

Yes.

Allowed Roles

Facility Manager, Manager, Admin.

Request Headers

```
{
  "Authorization": "Bearer JWT_TOKEN"
}
```

Success Response

```
{
  "message": "Workers retrieved successfully",
  "workers": [
    {
      "id": "86745ed3-f2f2-4de6-aae1-c1942d7e6a4c",
      "name": "Worker test",
      "email": "worker1@giu.edu.eg",
      "role": "worker"
    }
  ]
}
```

Error Response

```
{
  "error": "Access denied. Your role is not allowed to perform this action."
}
```

4.2.7 Common Error Responses

Missing or Invalid Token

```
{
  "error": "Unauthorized"
}
```

Forbidden Role

```
{
  "error": "Access denied. Your role is not allowed to perform this action.",
  "requiredRoles": ["manager", "Admin"],
  "yourRole": "worker"
}
```

Invalid Request Body

```
{
  "error": "Missing required fields"
}
```

Server Error

```
{
  "error": "Internal server error"
}
```

4.2.8 API Security Summary

The backend uses JWT authentication to secure protected routes. After login, the user receives a JWT token. This token must be included in the Authorization header for all protected endpoints.

Role-Based Access Control is applied using backend middleware. This ensures that:

- Community Members can only submit and view their own issues.
- Facility Managers can view all issues, assign workers, update statuses, and close issues.
- Workers can only view assigned issues, update progress, add comments, and upload completion photos.
- Unauthorized roles receive a 403 Forbidden response.